

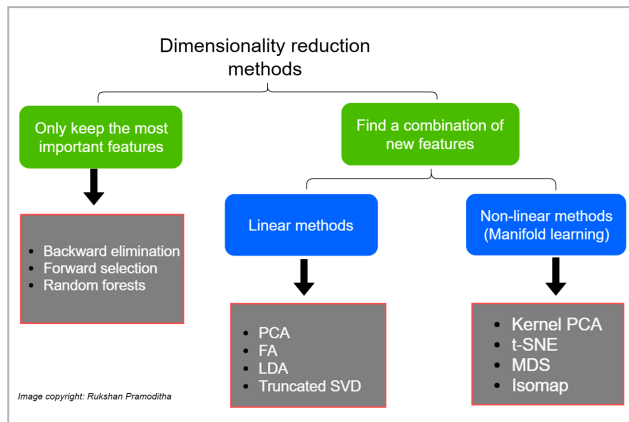


Figure 1: Techniques of dimensionality reduction

Principal component analysis (PCA)

PCA performs orthogonal transformation of the data such that the correlation between the data is minimised or zero, and the variance is maximised. The basis of the transformation is computation of Eigen values for the data.

The principal components are formed by the following rules:

- The translations are performed such that the centre of the data passes through the origin.
- The rotation is done such that the variance is maximised.

Singular value decomposition is used to find the principal components. These components are orthogonal to each other. Principal components are a linear combination of the original attributes.

Let's take an example of PCA with the wine data set from the scikit-learn library. Load the wine data set from *sklearn*:

```
from sklearn.datasets import load_wine
data = load_wine()
```

Scale the data so that the data is normalised:

```
from sklearn.preprocessing import StandardScaler
import pandas as pd
features = ['Alcohol', 'Malic acid', 'Ash', 'Alcalinity of ash', 'Magnesium', 'Total phenols', 'Flavanoids', 'NonFlavanoid phenols', 'Proanthocyanins', 'Color intensity', 'Hue', '00280/0D315 of diluted wines', 'Proline']
```

```
#Separating out the features
x = pd.DataFrame(data.data)
# Separating out the target
y = pd.DataFrame(data.target.columns = ['target'])
```

```
# Standardizing the features
x = StandardScaler().fit_transform(x)
```

Perform PCA as follows:

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
principalComponents = pca.fit_transform(x)
principalDf = pd.DataFrame(data = principalComponents, columns = ['principal component 1', 'principal component 2'])
```

Visualise the data with the plots:

```
finalDf = pd.concat([principalDf, y], axis = 1)
finalDf.head()

fig = pyplot.figure(figsize = (7,7))
ax = fig.add_subplot(1,1,1)
ax.set_xlabel('Principal Component 1', fontsize = 15)
ax.set_title('2 component PCA', fontsize = 20)
targets = [0, 1, 2]
colors = ['r', 'g', 'b']
for target, color in zip(targets, colors):
    indicesToKeep = finalDf['target'] == target
    ax.scatter(finalDf.loc[indicesToKeep, 'principal component 1'], finalDf.loc[indicesToKeep, 'principal component 2'], c = color, s = 50)
ax.legend(targets)
ax.grid()
```

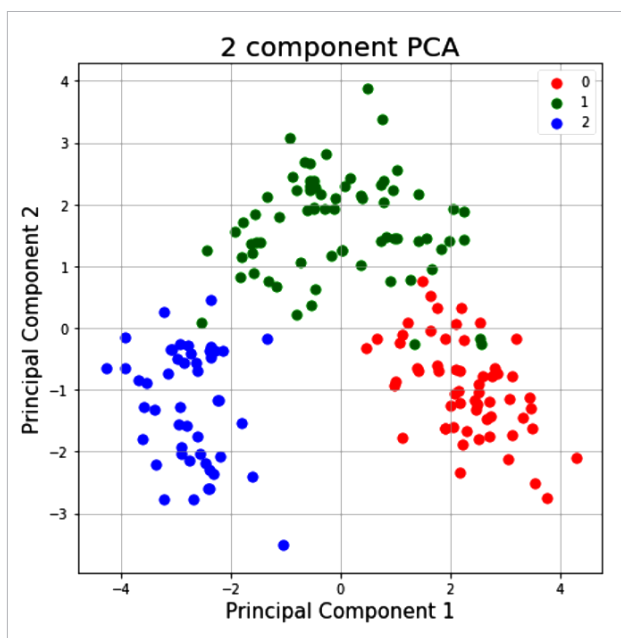


Figure 2: PCA for wine data set